

CertificateGSLT: Compositional Proof Checking over Validated GSLT Presentations

Open Derivations, DAG Evidence, and Multisorted-Clone
Semantics

(DRAFT --- August 25, 2026)

Oruži Zarathustra Goertzel

August 25, 2026

Abstract

A CertificateGSLT is a validated inference presentation — a finite family of judgment forms and generating rules layered over a validated **LanguageDef** syntax — regarded as a finite presentation of a proof-relevant multisorted theory. We present a Lean 4 formalization of the category theory and executable proof checking of such presentations. The development contains: a strict rule-retaining preorder of presentations whose arrows transport derivations, checked proofs, and even the identical chronological proof-DAG artifact; a compiled counterexample showing that strict rule retention cannot express every legitimate translation; a calculus of *open derivations* with positional premise occurrences and a proved simultaneous-substitution structure (both unit laws and associativity); a provably non-thin category of derivation-valued interpretations, in which one primitive rule may be implemented by a composite proof; set-valued models with contravariant pullback and a semantic-transport theorem; covariant indexed derivation families and a contravariant model family, each with a concrete category of elements; fuel-free checkers for untrusted open proof trees and chronological proof DAGs with premise holes, both reconstructing *exact* typed derivations; and an instantiation of a proof-relevant multisorted clone — the algebraic core of a cartesian multicategory — by the open derivations of every presentation. A compiled sharing canary separates compact DAG identity from expanded proof meaning: two submitted rule nodes expand to three rule occurrences. On top of this boundary we construct sharing-preserving substitution of checked artifacts: checked DAGs plug into the premise holes of a checked DAG so that acceptance is preserved, expansion commutes with composition exactly, and submitted node count is exactly additive while expanded cost multiplies per citation. All results are checked with no **sorry** and no custom axioms; the central theorems report exactly the standard axioms **propext**, **Classical.choice**, and **Quot.sound**. The framework also contains a proof-carrying direct trace generator; two-sided **Step/Steps** adequacy against the declarative MeTTaIL step relation, proved both on

a fixture and *generally* over an executable admitted fragment each of whose gate conditions carries a compiled counterexample showing the direct translation fails without it; a versioned chronological-article wire semantics with round-trip, canonicity, digest-identity, exact checker-correspondence, and conservative-evolution theorems, plus a compiled mutation battery; a genuine fixed-core binary coproduct of rule theories; exact certificate lower bounds with budget monotonicity; and the two orders of Belnap’s four evidence statuses. These results are stated at their proved scope: the adequate trace fragment is sufficient rather than maximal, the article version names the artifact grammar rather than its presentation, and the development does not claim a literal arithmetical-hierarchy classification, Patterson’s full constructible-duality logic, or a colimit of all languages.

1 Introduction

A validated `LanguageDef` finitely presents a language: sorts, constructors, binders, equations, and rewrite rules. Its *generated syntactic labelled transition system* (GSLT) is the derived rewriting theory — terms, equations, steps — and downstream tooling (parsers, rewrite engines, behavioral type systems) is *derived* from the one presentation rather than authored separately. The companion formalization `leanOSLF.tex` develops this pipeline for operational semantics. This paper develops the proof-theoretic lane of the same programme: as the rewrite lane generates a transition system from rewrite rules, the proof lane generates *derivations* from inference rules, and a `CertificateGSLT` is that generated derivational theory.

Many proof formats of interest — Metamath-style rule databases, MM0-style verified stacks, MeTTa-hosted proof calculi, and the rule sets authored in GSLT presentations themselves — share one shape: a finite list of judgment forms, a finite list of schematic inference rules, and untrusted proof artifacts that cite those rules. The engineering temptation is to build one checker per format. The `CertificateGSLT` thesis is that a single generic checker over validated presentations suffices, and that the *relationships* between presentations (extension, refinement, interpretation, semantic realization) form categorical structure worth proving once, at the framework level.

Concretely, a `CertificateGSLT` is presented by a validated inference presentation: judgments and rules are additional fields of the same authoritative `LanguageDef` that carries the syntax, so there is no second syntax authority, and judgments are ordinary `Pattern` terms of the shared syntax IR. The generic checker consumes exactly this data. Everything in this paper is stated and proved against that boundary.

1.1 Contributions

All contributions are Lean 4 theorems and executable definitions in the `Mettapedia/GSLT/LanguageDef/` tree; §22 gives the verification statement.

1. **Strict refinement.** A thin category of presentations whose arrows retain every source rule exactly. Arrows transport typed derivations, checked proofs, and accepted chronological proof DAGs *as identical artifacts*. A compiled negative theorem shows missing rules make arrows uninhabited (§3).
2. **A compiled inadequacy counterexample.** A two-presentation fixture in which the source rule $A \vdash C$ has no strict image, yet a derivation-valued interpretation implements it by a genuine two-node target proof (§4).
3. **Open derivations.** A proof-relevant calculus of derivations with typed, *positional* premise holes, with proved simultaneous substitution: two distinct unit laws, associativity, and exchange with positional lookup (§5).
4. **A non-thin interpretation category.** Interpretations map each admitted source rule application to a target open derivation over the same ordered premises. Composition recursively translates and plugs. The category laws follow from the substitution calculus, and a compiled witness shows a hom-type with two extensionally different arrows (§6).
5. **Clone semantics.** The open derivations of every presentation form a proof-relevant multisorted clone — the algebraic core of a cartesian multicategory: judgments are sorts, open derivations are operations, premise occurrences are projections, and proof plugging is simultaneous substitution (§7).
6. **Set-valued models.** Presentations are intensional; models realize them by judgment-indexed carriers and rule actions. Closed and open denotation, a semantic substitution lemma, contravariant model pullback along interpretations, and a semantic-transport naturality theorem are proved (§8).
7. **Indexed structure.** Covariant derivation and open-derivation functors, a contravariant model functor, and their concrete Mathlib categories of elements with projection functors — the precise sense in which derivations are the fibers of `CertificateGSLT` (§9).
8. **Executable evidence.** Fuel-free checkers for untrusted open proof trees and chronological open proof DAGs (premise references plus backward node references), with *exact* reconstruction: acceptance yields a typed open derivation whose erasure is the submitted artifact, and every erasure is accepted (§10, §11).
9. **The sharing boundary.** Strict refinement transports the identical DAG artifact; a compiled canary with two submitted nodes and three expanded rule occurrences keeps DAG identity and cost permanently distinct from expanded proof meaning (§12).

10. **Sharing-preserving artifact substitution.** Checked DAGs substitute into the premise holes of a checked DAG: the composite is accepted, expansion commutes with composition exactly, submitted node count is exactly additive while expanded cost multiplies per citation, and two compiled refuters reject the unshifted and uniformly wired variants (§13).
11. **A fail-closed trace generator and an operational weld.** Unsupported premise kinds cannot be silently erased: a `DirectTraceLanguage` carries the premise-fragment proof required by `stepPresentation`. A compiled counterexample rejects free congruence, and a nontrivial fixture proves two-sided one-step and trace adequacy against MeTTaIL’s declarative reduction, including the proof-relevant diamond correspondence (§14).
12. **General trace adequacy over a certified fragment.** Two-sided `Step/Steps` adequacy holds for *every* admitted language passing an executable gate: sequentially moded, hole-skeleton, congruence-only, non-reflective rewrite rules with unique names. Compiled counterexamples for three independent failure modes show the translation fails when any one of those gates is removed; the fragment is sufficient and is not claimed maximal (§15).
13. **A versioned article wire semantics.** Closed chronological proof DAGs receive a canonical symbolic encoding with round-trip, canonicity, and digest-identity theorems; a fail-closed versioned checker; and an exact correspondence — some accepted article claims a goal iff the goal is derivable, with linearization discharging the tree-versus-DAG replay obligation. A mutation battery rejects every structural and semantic tamper the format must exclude, and a conservative-evolution law transports the identical article along strict rule retention, with its converse compiled as a refutation (§17).
14. **Exact finite-certificate and budget results.** For every validated presentation, derivability is equivalent to existence of a raw proof accepted by the Boolean checker. On the adequacy fixture, a tower family has accepted certificates whose rule-node count is bounded below by the tower height, and a checked example proves that budget exhaustion is not refutation (§16).
15. **Fixed-core coproducts and paired evidence.** Validated, identifier-disjoint rule tables over one language core form a Mathlib binary coproduct in the strict category. Independent positive and negative certificate lanes yield Belnap’s four evidence statuses, both Belnap orders, and status-level strong negation; a database fixture inhabits all four cells and separates absent ex falso from an authored ex-falso instance (§18, §19).

1.2 Position in the formalization suite

leanOSLF ----- categorical GSLT foundations (syntax, OSLF types)

```

|
v
CertificateGSLT ----- generic proof-checkable semantics of presentations
|   |
|   +-----> MeTTaIL companion (LanguageDef, engines, instances)
|   +-----> Metamath / MMO instantiation targets
|
+-----> CeTTa / Prime proof packs, MettaKernel curriculum

```

2 Validated proof presentations

The trusted boundary is the generic inference checker (`Mettapedia/GSLT/LanguageDef/InferenceChecker.lean`). A `Presentation` is a derived view of the proof-calculus fields of the authoritative `LanguageDef`: a list of judgment declarations (outer head and arity) and a list of `RuleSchema` values, each with declared metavariables, ordered premises, and one conclusion, all over the shared `Pattern` syntax.

Validation is two-stage. V1 fixes the instantiation discipline: every metavariable has one exact occurrence depth, instantiation replaces only declared free metavariables at that depth, collection-rest metavariables are rejected, and binder metadata must be canonical — no implicit lifting is performed. V2 adds the finite structural judgment signature: every premise and conclusion resolves against a declared judgment head and arity, and fixed nested constructor occurrences resolve against the underlying `LanguageDef`. A `ValidatedPresentation` is a presentation with `isValidV2 = true`, and

```

structure Object where
  presentation : ValidatedPresentation

```

is the object of every category in this paper (`Mettapedia/GSLT/LanguageDef/CertificateGSLT.lean`).

Three layers of proof data sit over an object, from trusted to untrusted:

Layer	Lean form	Status
Typed derivation	<code>Derivation P goal</code>	trusted, <code>Type</code> -valued
Checked proof	<code>CheckedProof P goal</code>	raw tree + acceptance proof
Raw artifact	<code>RawProof</code> , DAG blocks	untrusted wire data

`RuleApplication P i ps c` is the checker’s evidence that rule instance `i` admits ordered premises `ps` and conclusion `c`; it is equivalent to successful executable instantiation (`instantiateRule?.eq.some.iff_application`), which is the hinge between the trusted and executable layers throughout.

3 Strict refinement: the rule-retaining preorder

The first category is deliberately austere. A strict arrow retains every source rule with the same identifier and schema:

```
structure Morphism (source target : RuleRetaining) : Type where
  refines : RuleLookupRefines source.presentation target.presentation
```

Strict arrows are proof-irrelevant (`Subsingleton`), so `RuleRetaining` is a *thin* category — a preorder. This thinness is not an accident but a theorem-shaped statement about checking: an arrow exists exactly when every source rule lookup succeeds identically in the target, and then everything the checker accepted before is accepted after. Three transport results make that precise:

- `mapDerivation` : typed derivations transport along `refines`;
- `mapCheckedProof` : checked proofs transport *retaining the exact raw tree* (`mapCheckedProof_payload` is `rfl`);
- `CheckedDAG.transportMeaning` : an accepted chronological proof DAG re-expands to the same raw proof, which the target accepts, with a transported typed derivation — the naturality square of the generic DAG checker.

Validated append-only presentation extensions embed as strict arrows (`ofValidatedExtension`). On the negative side, the category does not degenerate into an indiscrete preorder:

Theorem 1 (`hom_isEmpty_of_missing`). *If the source presentation binds a rule identifier that the target does not, the strict hom-type is uninhabited.*

4 Why strict retention is not enough

Strict refinement cannot express translations that *implement* a rule rather than copy it. The compiled fixture (`Mettapedia/GSLT/LanguageDef/CertificateGSLTInterpretationCanary.lean`) has a source presentation with one rule $A \vdash C$ and a target presentation with rules $A \vdash B$, $B \vdash C$, and a two-premise sharing rule $B, B \vdash C$ — but no rule named by the source identifier.

Theorem 2 (`strict_rule_retaining_interpretation_is_empty`). *The strict hom-type from the source to the target is uninhabited.*

Theorem 3 (`general_interpretation_exists, target_implementation_has_two_rule_nodes`). *A derivation-valued interpretation from source to target exists, and it implements the source rule by a target proof with exactly two primitive rule nodes ($B \vdash C$ applied to $A \vdash B$).*

This pair of theorems makes open proof templates load-bearing: any adequate category of proof-system translations must let a primitive rule map to a composite proof, and composite proofs need typed premise holes.

5 Open derivations and simultaneous substitution

An *open derivation* $\Gamma \vdash_P A$ derives judgment A from an ordered context Γ of premise *occurrences* (Mettapedia/GSLT/LanguageDef/CertificateGSLTOpen.lean):

```
inductive OpenDerivation (P : ValidatedPresentation)
  (context : List Pattern) : Pattern -> Type where
  | assumption (index : Fin context.length) :
    OpenDerivation P context (context.get index)
  | byRule (i : RuleInstance) (application : RuleApplication P i ps c)
    (children : OpenDerivationList P context ps) :
    OpenDerivation P context c
```

Premise holes are addressed by position (**Fin**), not by judgment: two equal judgments at different positions are different occurrences. A position may be cited never, once, or repeatedly, so weakening and contraction are present by construction rather than postulated. This positional discipline is exactly the cartesian structure that §7 names.

An *environment* $\sigma : \Delta \vdash_P \Gamma$ is an ordered vector of open derivations, one per position of Γ . Substitution `bind` plugs an environment into every hole. The proved laws, writing $d[\sigma]$ for `d.bind` σ and id_Γ for the identity environment of premise assumptions:

$$\begin{array}{ll}
 x_i[\sigma] = \sigma_i & (\text{assumption_bind, get_bind}) \\
 d[\text{id}_\Gamma] = d & (\text{bind_assumptionEnvironment}) \\
 \text{id}_\Gamma[\sigma] = \sigma & (\text{assumptionEnvironment_bind}) \\
 d[\sigma][\tau] = d[\sigma[\tau]] & (\text{bind_assoc})
 \end{array}$$

The two unit laws are deliberately distinct statements: the right unit says substituting the identity environment into a derivation changes nothing; the left unit says plugging any environment into the vector of premise assumptions returns that environment.

Closed derivations embed into any context (`ofClosed`), open derivations over the empty context close (`close`), and the two maps are mutually inverse (`close_ofClosed`, `ofClosed_close`), so closed proof theory is literally the empty-context fragment.

6 The non-thin category of interpretations

Definition 4 (Interpretation). An interpretation $F : P \rightarrow Q$ maps every admitted source rule application with ordered premises $\vec{p}$ and conclusion c to a target open derivation $\vec{p} \vdash_Q c$.

Translation `mapOpen` is the evident recursion: premise occurrences map to themselves, and a rule node maps to the interpreting template with the translated children plugged into its holes. The substitution-naturality law

$$F(d[\sigma]) = F(d)[F(\sigma)] \quad (\text{mapOpen_bind})$$

is proved by mutual induction with associativity, and the category laws of

```
instance : Category Object where
  Hom := Interpretation
  id := Interpretation.id
  comp := Interpretation.comp
```

reduce to the unit and associativity laws of §5. Identity interprets each rule by itself over one distinct hole per premise; composition interprets, then recursively reinterprets. Strict arrows embed by `ofRuleLookupRefines`, interpreting each rule by its transported one-node template.

Unlike the strict preorder, this category is *provably* not thin. In the fixture of §4, the source rule also has a three-node implementation through the sharing rule $(B, B \vdash C$ applied to two copies of $A \vdash B$):

Theorem 5 (`parallel_interpretations_distinct`, `interpretation_hom_not_subsingleton`).

The two-node and three-node interpretations are distinct parallel arrows: evaluating both at the canonical source rule application yields implementations with primitive rule counts 2 and 3. Consequently the hom-type is not a subsingleton, so the interpretation category is not thin.

Remark 6. Proof identity, not merely provability, is the subject matter here. A thin quotient of this category would identify proof translations with different costs and different sharing behavior; §12 shows why that identification must not be baked in.

7 The internal calculus: a proof-relevant multi-sorted clone

The framework-level algebra of open derivations has a standard name. A *multisorted clone* over a sort set S assigns to every ordered context $\Gamma \in \text{List } S$ and sort A a type of operations $\text{Hom}(\Gamma, A)$, with positional projections and simultaneous substitution satisfying the projection, identity, and associativity laws (`Mettapedia/GSLT/LanguageDef/MultiSortedClone.lean`). Because positions may be dropped or repeated, this is the algebraic core of a *cartesian* multicategory, as opposed to the linear discipline of a plain multicategory.

Theorem 7 (`derivationClone`). *For every CertificateGSLT object, open derivations form a multisorted clone over **Pattern**: sorts are ground judgment patterns, operations are open derivations, projections are premise occurrences, and substitution is proof plugging. All clone laws are instances of the theorems of §5.*

The scope of this statement is deliberately structural. The clone supplies the context discipline shared by every presented proof system — projection, weakening, contraction, cut-as-substitution. Logical connectives, binders, conversion rules, and side judgments are *content of the presented object theory*: they live in the rules of a particular presentation, not in the framework. `CertificateGSLT` is in this sense a logical-framework-shaped theory whose internal calculus is the substitution structure itself.

8 Set-valued models and semantic transport

A presentation is intensional: it says how derivations are *generated*. A `Model` realizes it extensionally (`Mettapedia/GSLT/LanguageDef/CertificateGSLTSemantics.lean`): a carrier type for every judgment pattern, and an action of every admitted rule application sending realizations of the ordered premises to a realization of the conclusion.

Closed derivations fold into any model (`denote`); open derivations denote functions of a premise environment (`denoteOpen`). The semantic substitution lemma interprets plugging as composition:

$$\llbracket d[\sigma] \rrbracket_\rho = \llbracket d \rrbracket_{\llbracket \sigma \rrbracket_\rho} \quad (\text{denoteOpen.bind})$$

Models pull back *contravariantly* along interpretations: given $F : P \rightarrow Q$ and a Q -model M , the P -model F^*M keeps the carriers and interprets each source rule by the denotation of its interpreting template. The functor laws `pullback_id` and `pullback_comp` are proved, and semantic transport is a naturality theorem:

Theorem 8 (`denote_mapDerivation`). $\llbracket F(d) \rrbracket_M = \llbracket d \rrbracket_{F^*M}$ for every closed source derivation d and target model M ; likewise for open derivations under any environment (`denoteOpen_map`).

Interpretations are therefore sound for arbitrary set-valued semantics: translated proofs mean what the pulled-back model says the originals mean.

9 Indexed structure and categories of elements

The slogan “derivations are the fibers of `CertificateGSLT`” is made precise by three functors (`Mettapedia/GSLT/LanguageDef/CertificateGSLTIndexed.lean`):

- `derivationFunctor goal : Object → Type`, covariant, acting by `mapDerivation`;
- `openDerivationFunctor context goal`, covariant, acting by `mapOpen` and retaining the ordered premise context;
- `modelFunctor : Objectop → Typeu+1`, contravariant, acting by `pullback`.

Covariance of derivations and contravariance of models is the expected variance split of a theory/model duality. Context substitution inside one fiber is the separate axis supplied by `bind`, and the two axes commute (`bind_natural`).

Each functor has its concrete Mathlib category of elements — `DerivationTotal`, `OpenDerivationTotal`, `ModelTotal` — with projection functors to the base. An object of `DerivationTotal goal` is a presentation together with one closed proof of `goal`; a morphism is an interpretation whose action sends the source proof to the target proof exactly. These totals are the concrete Grothendieck-construction endpoints over which a fully displayed organization (over compositional syntax translations) will later sit.

10 Executable evidence I: open proof trees

Typed open derivations are the trusted calculus; wire artifacts are untrusted (`Mettapedia/GSLT/LanguageDef/CertificateGSLTOpenChecker.lean`):

```
inductive RawOpenProof where
| premise (index : Nat)
| node (ruleInstance : RuleInstance) (children : List RawOpenProof)
```

The checker `checkOpenRaw` is structurally recursive — fuel-free — and fails closed on out-of-range premise references and failed rule instantiations. Its adequacy is exact in both directions:

Theorem 9 (`checkOpenRaw_erase`; `checkOpenRaw_exact_derivation`). *The erasure of every typed open derivation is accepted; and every accepted raw tree is the erasure of some typed open derivation of the same goal in the same context.*

Corollary 10 (`checkedOpenProof_nonempty_iff_openDerivation`). *Checked open evidence exists if and only if a typed open derivation exists.*

“Exact” matters: acceptance does not merely assert provability, it reconstructs a derivation whose erasure is *the submitted artifact*, so cost and shape claims about artifacts transfer to typed proofs.

11 Executable evidence II: chronological proof DAGs

Compact evidence shares subproofs (`Mettapedia/GSLT/LanguageDef/CertificateGSLTOpenDAG.lean`). A chronological DAG is a list of blocks of nodes; each node carries a fresh identifier, a rule instance, and children that either cite an authored premise position or an *earlier* derived node:

```
inductive OpenDAGReference where
| premise (index : Nat)
| node (id : Nat)
```

Duplicate identifiers are rejected; the chronological environment makes every node reference backward by construction, so no separate acyclicity proposition is trusted. Checking maintains a ghost expansion: each accepted node stores the exact `RawOpenProof` tree it denotes.

Theorem 11 (`checkOpenDAGBlocks_exact_derivation`). *An accepted DAG expands to an exact raw open proof accepted by the tree checker, hence to a typed open derivation with that exact erasure.*

A compiled negative canary (`missing_premise_rejects`) shows an out-of-range premise reference failing closed. The closed-proof analogue of this layer (checker `checkDAGBlocks`, used in §3) also has an untrusted bottom-up hash-consing compiler (`Mettapedia/GSLT/LanguageDef/InferenceProofDAGCompilation.lean`); its output is rechecked, never trusted.

12 Exact transport and the sharing boundary

Strict refinement preserves not only meaning but the artifact itself (`Mettapedia/GSLT/LanguageDef/CertificateGSLTDAGTransport.lean`): every successful source rule instantiation replays identically in the target, so node, block, and root computations are unchanged.

Theorem 12 (`checkOpenDAGBlocks_true_of_ruleLookupRefines; CheckedOpenDAG.transport`). *Under strict rule retention, the same blocks and root identifier are accepted by the target, and transport preserves them definitionally (`transport_rootId` and `transport_blocks` are `rfl`).*

General interpretations deliberately do *not* satisfy artifact transport: implementing one primitive node by a multi-node template changes the artifact. What they need instead is a genuine graph substitution, and its premise-hole core is constructed below (§13).

The reason sharing must be tracked separately from meaning is compiled. In the fixture, one derived *B* node is cited twice by the sharing rule:

Theorem 13 (`shared_open_dag_accepts; shared_open_dag_expands_exactly; sharing_changes_rule_occurrence_count`). *The two-node artifact is accepted, expands exactly to a tree that visibly duplicates the shared subtree, and the counts differ: the submitted DAG has 2 rule nodes while the expansion has 3 rule occurrences.*

Consequently DAG-node cost is not expanded-tree cost, and compact artifact identity is not proof-tree identity. The framework keeps three notions separate by construction: expanded proof meaning (the typed derivation), compact evidence identity (the DAG artifact), and any future cost or provenance decoration over it.

13 Sharing-preserving substitution of checked artifacts

Open derivations compose by `bind`; this section composes their compact evidence (`Mettapedia/GSLT/LanguageDef/CertificateGSLTDAGSubstitution.lean`, raw layer in `CertificateGSLTRawSubstitution.lean`). Given a checked DAG for $\Gamma \vdash A$ and one checked DAG per premise position of Γ , all over a common ambient context Δ , the composite (`CheckedOpenDAG.substitute`) lists every environment segment *once*, then the outer DAG with each premise citation rewired to the root of its segment. Identifiers are kept disjoint by shifting each segment past the identifier bound of everything before it; only the outer DAG's premise citations are rewired, while segments keep their ambient citations.

Two compiled refuters pin this design against its natural competitors (`CertificateGSLTDAGSubstitutionCa.lean`): composing without the shift collides identifiers and is rejected rather than silently merging distinct rule nodes (`unshifted.composition.rejected`), and wiring segment citations uniformly like outer holes makes a node cite itself before it exists and is rejected (`selfwired.environment.rejected`).

The surviving design satisfies the three laws that make it the artifact presentation of clone substitution. Write $D[\vec{E}]$ for the composite and $\widehat{D}$ for the ghost expansion of an accepted artifact.

Theorem 14 (`substituteDAGBlocks.accepts`; `substituteDAGBlocks.expands`; `CheckedOpenDAG.substitute_expansion`). *The composite is accepted, and expansion commutes with composition exactly: $\widehat{D[\vec{E}]} = \widehat{D}[\widehat{E_1}, \dots, \widehat{E_n}]$, where the right-hand side is raw substitution of proof trees.*

Theorem 15 (`OpenDerivation.eraseOpen_bind`; `CheckedOpenDAG.substitute_erases_bind`). *Erasure is a homomorphism from typed plugging to raw substitution, so whenever the constituents erase typed open derivations, the composite's expansion is the erasure of the plugged derivation: artifact substitution presents the clone operation of §7.*

Theorem 16 (`substituteDAGBlocks.nodeCount`; `RawOpenProof.substitute_ruleCount`; `CheckedOpenDAG.substitute_expansion_ruleCount`). *Submitted node count is exactly additive, $|D[\vec{E}]| = |D| + \sum_i |E_i|$, no matter how often a position is cited; while the expanded tree pays each segment once per citation: $\text{rc}(\widehat{D[\vec{E}]}) = \text{rc}(\widehat{D}) + \sum_i c_i \cdot \text{rc}(\widehat{E_i})$, with c_i the citation count of position i in $\widehat{D}$.*

On the two-node fixture the packaged operation reproduces the boundary of §12 from the general theorems alone: compact cost 2, expanded cost 3 (`packaged.composition.counts`).

Two honest boundaries remain by design. Substitution never re-shares across constituents — structurally equal nodes from different segments keep distinct identifiers; re-hash-consing is a separate, untrusted optimization whose output is rechecked. And the artifact calculus has no identity artifacts: a root must be a rule node, so identity environments and the unit and associativity laws live at the expansion level, where they are exactly the clone laws.

14 Direct trace presentations and the OSLF weld

A computational step relation is one important proof theory, but operational semantics does not generally grant contextual closure for free. In MeTTaIL, a rewrite may use a substep only when its authored premise is a `Premise.congruence`; quoted data, suspended continuations, and lazy cells can therefore remain inert. The first version of the trace generator incorrectly emitted congruence at every constructor position. The compiled counterexample `congruence_is_not_free` exhibits a language with $a \rightarrow b$ but no step $f(a) \rightarrow f(b)$, permanently excluding that convention.

The corrected generator is fail-closed at the premise boundary. A `DirectTraceLanguage` packages a `LanguageDef` with evidence that every rewrite premise belongs to the directly translated congruence fragment. Only this proof-carrying source is accepted by `stepPresentation`; a language containing a relation-query premise is rejected by `DirectTraceLanguage.ofLanguage?` rather than being translated after silently dropping the premise. The generated theory has two judgments,

$$\text{Step}(s, t), \quad \text{Steps}(s, t),$$

one `Step` schema per authored rewrite, and reflexive–transitive trace rules for `Steps`. Generated-presentation validation remains a separate gate: it checks metavariable depths, collection-rest freedom, constructor references, and judgment signatures.

The collection-rest theorem is another exact format boundary. `isValidV2.eq_false_of_collectionRest.ru` proves that a schema containing a bag-rest pattern cannot enter the current checker. This does not place bag semantics beyond `CertificateGSLT`: one may present decomposition and exchange as explicit inference, or extend the checker with a verified bag-decomposition certificate. What is ruled out is an unrecorded matching choice inside the existing schema format.

The source-adequacy result was proved first as an *instance theorem* on a fixture containing a ground rewrite and an authored congruence rule; §15 lifts it to every language of an executable fragment, and the fixture remains the concrete anchor for the modal weld below. For well-formed sources,

$$\begin{aligned} \text{Derivation}(\text{Step}(s, t)) \text{ inhabited} &\iff s \text{ langReduces } t, & (\text{step_adequacy}) \\ \text{Derivation}(\text{Steps}(s, t)) \text{ inhabited} &\iff \text{ReflTransGen}(\text{langReduces})(s, t). & (\text{steps_adequacy}) \end{aligned}$$

The proof handles both directions against the actual matcher and premise semantics. It yields the first proof-relevant OSLF square on the same fixture:

$$\Diamond \varphi(s) \iff \exists t, \text{CheckedStep}(s, t) \wedge \varphi(t).$$

This is `diamond_iff_checked_witness`. The box theorem presently proves only the sound direction for checked, well-formed predecessors; finite-predecessor evidence or an authored invariant theory is needed for a converse.

15 General trace adequacy over a certified fragment

The fixture theorem generalizes, but the statement that does so must be chosen carefully: two-sided adequacy is *false* for arbitrary admitted `DirectTraceLanguages`, and each failure mode is now a compiled counterexample rather than a design suspicion. The general theorems therefore quantify over an executable admission gate, `languageDirectTraceAdequate`, chosen so that everything inside it is proved and nothing outside it is constrained.

Definition 17 (adequate direct-trace fragment). *A language passes the gate when its rewrite rules satisfy:*

1. sequential modedness — *reading premises in authored order, every congruence source is bound by the left-hand side and earlier congruence targets, and the right-hand side is bound by the end of the chain;*
2. hole-skeleton patterns — *rule patterns are constructor/binder skeletons whose metavariables occur at ambient depth only, with no explicit-substitution and no collection nodes;*
3. *every premise is an authored congruence premise (the `DirectTraceLanguage` gate);*
4. *the language declares no reflective presentations, and rewrite names are unique.*

Theorem 18 (`directTrace_step_adequacy`, `directTrace_steps_adequacy`). *Let L pass the gate and let its generated presentation be admitted. For checker-well-formed sources s ,*

$$\begin{aligned} \text{Derivation}(\text{Step}(s, t)) \text{ inhabited} &\iff s \text{ langReduces}_L t, \\ \text{Derivation}(\text{Steps}(s, t)) \text{ inhabited} &\iff \text{RefTransGen}(\text{langReduces}_L)(s, t), \end{aligned}$$

and declarative steps preserve checker well-formedness. Composing with §16, one-step reduction and reachability in L are each equivalent to existence of an accepted finite certificate over the generated presentation.

The proof aligns the two executable semantics that meet in the generated schemas. Soundness inverts an arbitrary accepted rule application into the language’s own matcher: a match-own-instance lemma produces matcher bindings from checker arguments, a merge algebra for `mergeBindings` carries binding agreement through the premise chain, and a fuel-monotonicity theorem for the contextual step relation lets independently obtained premise witnesses share one rule application. Completeness runs the alignment in reverse, reading checker arguments off the engine’s final bindings; the exact bridge is a proved agreement between schema instantiation and binding application on hole-skeleton patterns.

Why the present gates are sufficient. The canary module supplies compiled counterexamples for three independent failure modes of the direct translation, each showing that deleting that gate alone breaks it:

- **moded_gate_necessary:** for the rule $unit \rightarrow x$ with x unbound by the left-hand side, the generated presentation is admitted and certifies the ground step $\text{Step}(unit, unit)$, yet the language steps $unit$ only to the free variable — soundness fails;
- **subst_gate_necessary:** an explicit-substitution reduct is evaluated by binding application but preserved structurally by schema instantiation, so the checked target is not the declarative target;
- **bag_gate_necessary:** collection matching is multiset-commutative while schema instantiation is positional; the declarative step $\{unit, think\} \rightarrow \text{pair}(think, unit)$ through a permuted match provably has no certificate — completeness fails;
- relation-query premises remain rejected both by `DirectTraceLanguage.ofLanguage?` and by the gate.

These counterexamples calibrate the claim precisely, and their scope should be read exactly. The non-reflective hypothesis is used essentially by the present proof, which identifies source matching and application with the syntactic checker operations; replacing it requires a separately proved reflective-matcher adequacy interface, and no counterexample is offered for it. Rewrite-name uniqueness is a locally checkable convenience already implied by validity of the generated presentation — duplicate rewrite names produce duplicate generated rule identifiers — and is therefore not an independent semantic gate. What is established is that the fragment is *sufficient* and that the three exhibited axes are load-bearing for this particular direct translation; neither maximality nor translation-independent necessity is claimed. Richer translations remain open where they arrive with more machinery: a verified rewrite-modulo certificate could readmit collection patterns, and certified premise translations could admit relation and freshness premises (§23). The formalization constrains exactly the proved boundary and nothing else.

The positive canary exercises the features the fixture lacked: a language whose reduct carries a binder literal, and a congruence rule discharging *two* authored premises in one application. Its certificates are obtained through the general theorem rather than by a hand-built fixture proof, and a compiled negative confirms that the generated presentation cannot certify a step the language does not take.

16 Finite certificates, lower bounds, and budgets

At the checker boundary, a general theorem requires no operational fixture:

Theorem 19 (`derivation_nonempty_iff_certificate`). *For every validated presentation P and goal A ,*

$$\text{Nonempty}(\text{Derivation}_P(A)) \iff \exists \pi : \text{RawProof}, \text{checkRaw}(P, A, \pi) = \text{true}.$$

The right side has the familiar finite-certificate shape of a semidecidability result. We do not call it a formal Σ_1^0 classification: that stronger statement additionally requires a natural number coding and effective enumeration of `RawProof`.

The adequacy fixture supplies a quantitative family. The n -fold contextual lift $f^n(a) \rightarrow f^n(b)$ is proved uniformly by one induction (`tower_reachable`); every member has an accepted certificate; and `tower_certificate_lower_bound` proves that every accepted certificate contains at least $n + 1$ rule nodes. Thus the development proves both uniform derivability and an unbounded family of certificate-size lower bounds without identifying those two kinds of evidence.

`checkWithinBudget` adds a two-valued verification status: `established` or `undetermined`. Established results are sound and monotone under increasing budgets. The theorem `budget_exhaustion_is_not_refutation` gives a true step for which every certificate at budget one is undetermined. This is the proved budget core; a full runtime verdict may add separately checked negative evidence and operational interruption, but those are not constructors of this datatype.

17 Versioned chronological articles: a wire semantics

The DAG checker of §11 fixes what evidence *means*; this section fixes how evidence *travels*. A `WireArticle` is the closed-proof interchange artifact:

```
WireArticle
|- version -- ABI version (currently 1)
|- nodes -- chronological DAG nodes
| |- node identifier
| |- authored rule identifier
| |- exact rule arguments, authored order
| |- ordered child references
|- root node identifier
|- claimed target judgment
```

Chronology is load-bearing: a node may cite only strictly earlier nodes, so cycles, self-references, and forward references fail locally, without a global occurs check. The carrier is a symbolic S-expression layer (`WireTerm`); byte framing of that layer is deliberately below this ABI.

Encoders are total and decoders fail closed. Three theorems make the encoding an identity discipline rather than a serialization convenience: *round trip* (`decodeArticle` of `encodeArticle` returns the article); *canonicity* (any term that decodes at all is the canonical rendering of its value, so each article has exactly one wire form); and *injectivity* (`encodeArticle_inj_iff`: equal canonical

renderings are equal articles). This is injectivity on articles, not a cryptographic digest theorem, and it identifies no presentation; it is the layer on which a content hash could later be built.

`checkWireArticle` composes a fail-closed version gate with the chronological DAG checker of §11 at the empty premise context. Its correspondence is exact in both directions:

Theorem 20 (`wireArticle_correspondence`). *For every validated presentation P and goal A : some accepted article stores target A if and only if $\text{Derivation}_P(A)$ is inhabited.*

Soundness reconstructs a closed derivation from any accepted article via exact reconstruction and the empty-context closing bridge. Completeness is a linearization theorem: `Derivation.linearize` emits a chronological node list with sequential fresh identifiers, and an environment-invariant induction proves the resulting article accepted with the derivation’s goal as its stored target. Linearization also discharges the tree-versus-DAG replay obligation: tree-shaped proofs enter the ABI as their linearizations, and accepted articles expand back to exact trees.

A mutation battery accompanies the positives. Over a small authored presentation, one three-node article is accepted — directly and through its rendering — and compiled negatives reject: a wrong ABI version; an unknown rule identifier; a non-ground argument; a duplicate node identifier; forward and self references; omitted, extra, and *swapped* children; a rule-identifier tamper; a premise reference in a closed article; a bad root; a target mismatch; an undecodable term; and a tampered rendering checked through decode. The swapped-children rejection is the ordered-bag discriminator: premise order is semantic, and the wire format does not treat proof children as a multiset.

Exact-rule artifact retention. Articles outlive the presentation revision that produced them, so one reuse law is proved directly at the wire boundary:

Theorem 21 (`checkWireArticle_true_of_ruleLookupRefines`). *If `RuleLookupRefines` holds from a source presentation to a target presentation, then every article the source accepts is accepted by the target — the identical article, with the same node identifiers, rule instances, child order, and sharing. The same transport holds through the symbolic decode boundary (`checkWireTerm_true_of_ruleLookupRefines`).*

The name matters. This is retention of *artifacts under exact rule lookup*: it is not semantic conservativity, and it says nothing about source models, source-language meaning, or equivalence of the two presentations. Strict rule retention is exactly the hypothesis that makes it unconditional: instantiation depends only on the looked-up schema and the argument vector, and retention preserves every successful lookup. The converse fails, and is refuted rather than merely omitted: `new_article_does_not_transport_backward` exhibits an article citing a rule added by an extension, accepted by the extension and rejected by the

base. Evidence therefore moves forward under conservative refinement and never backward.

This law covers exactly one mode of change. Rule deletion, rule-identifier reuse under altered semantics, and deliberate semantic revision are *not* covered, and should not be: a framework in which old evidence survived an intentional revision would be unsound as a checking discipline. Distinguishing conservative extension, semantics-preserving translation, and intentional revision — and attaching the right identity to each — is the subject of the following paragraph.

Local rule agreement, and article identity. Whole-table retention is more than an article needs and more than a large library can supply: an article cites a handful of identifiers out of a table that may hold tens of thousands. The exact hypothesis is agreement on the cited identifiers alone, and it yields a biconditional:

Theorem 22 (`checkWireArticle_iff_articleRuleAgreement`). *If two validated presentations resolve every identifier an article cites to the same schema, they accept exactly the same article.*

Whole-table retention factors through this, and the local form is what makes content-addressed article reuse across independently growing presentations possible.

One further property is *not* settled by the acceptance theorems above. The checker validates every node and then requires only that the root carry the target; it never requires nodes to be reachable from the root. A compiled witness confirms that an unreferenced but individually valid node leaves acceptance unchanged. This is not unsoundness — the root still reconstructs an exact derivation — but it means the submitted-node count and any content hash of an article are not functions of the proof, which the compact-cost ledger of §12 depends on. Reachability, its executable check, and the strengthened checker `checkRootedArticle` are therefore given, with soundness inherited unchanged; a compiled canary shows the strengthened checker keeps the intended article and rejects the padded one. Making rootedness an ABI *requirement* additionally owes the proof that linearised articles are always rooted, which is what would carry the completeness half across; until then it is an available strengthening rather than part of the format.

What the ABI does not yet fix. The version field identifies the artifact grammar and checking semantics only. The presentation is supplied externally to `checkWireArticle`; the article does not record which semantic presentation it belongs to. Before persistent or cross-process use, an article should be enveloped by a semantic identity — presentation digest, language identity, and policy version alongside the ABI version — so that a structurally valid article cannot be interpreted against the wrong theory. Likewise, `articleOfDerivation` is a straightforward linearization that does not compress repeated subproofs; sharing-optimizing encoders remain untrusted producers, free to emit any article the checker accepts.

18 Fixed-core coproducts of rule theories

Let P_L and P_R be validated rule tables over one syntax-and-judgment core, with disjoint identifiers, and suppose their append P_{L+R} is also valid. Exact lookup gives strict arrows

$$P_L \longrightarrow P_{L+R} \longleftarrow P_R.$$

Theorems `join_refines_left`, `join_refines_right`, and `join_refines_of_both` establish the preorder universal property. The construction is additionally packaged against Mathlib’s categorical interface:

Theorem 23 (`joinCofanIsColimit`). *The displayed binary cofan is a colimit in the strict rule-retaining category; equivalently, the append is a binary coproduct of these two objects.*

This is a genuine finite colimit theorem, but its scope is important. It is not yet a coproduct across different syntax translations, a universal interpreter, or a colimit of all computational systems. Those statements require the syntax-translating interpretation layer and a specified diagram.

19 Paired evidence and Belnap’s four statuses

The checker is neutral about negation. Nevertheless, two independently authored goals and two offered certificates form the evidence-status carrier used by constructible-duality, strong-negation, and paraconsistent systems:

	negative absent	negative accepted
positive absent	undetermined	refuted
positive accepted	established	conflicted

The equivalence `fourEquivLanes` proves that this table is exactly `Bool × Bool`, not merely a four-constructor analogy.

`FourVerdict.knowledgeLE` is Belnap’s approximation order, with undetermined below and conflicted above. `FourVerdict.truthLE` is the second order, with refuted below and established above. Both are proved reflexive, transitive, and antisymmetric. Status-level strong negation swaps established and refuted, fixes conflicted and undetermined, is involutive, preserves the knowledge order, and reverses the truth order. The operation `pairedVerdict` checks offered certificates and its soundness theorems reconstruct the positive derivation, negative derivation, or both, according to the resulting status. Conversely, erasures of supplied typed derivations produce the corresponding established, refuted, or conflicted status.

A database canary joins a `Holds` table with a `Denied` table and inhabits all four cells. Conflict at one atom does not derive an unrelated atom (`no_explosion`); appending one explicit two-premise rule derives that atom from the conflict (`explosion_is_authorable`). The result is not that the generic

checker is intrinsically paraconsistent. Rather, it adds no implicit ex-falso principle: contradiction propagation is determined by the authored rule theory.

This section formalizes an evidence carrier and its orders, not Patterson’s full logic of constructible duality [9]. A complete account still needs judgment-level strong negation, logical connectives, the lane-swapping interpretation, and appropriate semantics. Likewise, graded (n^+, n^-) or probabilistic evidence for PLN is a future refinement of the Boolean lanes, not a consequence of the present theorems.

20 Applications and instantiation targets

MeTTaIL-authored presentations. Because judgments and rules are fields of `LanguageDef`, every authored GSLT presentation that declares them is already a `CertificateGSLT` object; the companion `MeTTaIL.tex` describes the authoring pipeline. The framework then supplies, without further authorship: a proof calculus with substitution, an interpretation category, model pullback, and two checked evidence formats.

Metamath and MM0. Rule-database proof formats are the intended stress instantiations: their presentations are large but structurally uniform, and their proof artifacts are naturally chronological DAGs (Metamath’s compressed proofs already cite earlier steps by index). The design intent is that both instantiate the *unchanged* generic checker, with source-adequacy obligations (that the encoded presentation means the original database) kept in their respective front ends. This instantiation is designated future work (§23); the independent verified Metamath checker `mm-lean4` in the same repository is the natural conformance partner.

CeTTa and Prime proof packs. The CeTTa runtime consumes compiled proof packs. Compiling general `CertificateGSLT` interpretations and DAG evidence to that format, with a checker correspondence theorem, is the designated systems-facing exit; the hash-consing compiler of §11 is the existing seam, and the versioned article of §17 is the interchange contract a native replay checker consumes.

21 Related work

Logical frameworks in the LF tradition [1] represent object logics inside a dependently typed λ -calculus, obtaining substitution and binding from the framework. `CertificateGSLT` is deliberately weaker at the framework level: it fixes only the first-order structural core — positional contexts and simultaneous substitution — and leaves binders and conversion to the presented theory, in exchange for a small executable checker and artifact-exact reconstruction theorems. The little-theories method [2] develops mathematics as a graph of small theories connected

by interpretations; the non-thin category of §6 is a proof-relevant refinement of that picture in which an interpretation carries the proofs it performs. Clones and Lawvere theories [3] are the classical algebra of cartesian operations; multicategories originate with Lambek [4]. Metamath [5] demonstrates how far a minimal generic rule checker scales; MM0 [6] adds verified translation between proof languages. CertificateGSLT aims the same minimalism at the GSLT setting, with the category theory of presentations proved rather than implicit.

Meseguer’s rewriting logic [7] gives computations a proof-theoretic reading with reflexivity, congruence, replacement, and transitivity. The trace generator is close in form but intentionally does not inherit unrestricted congruence: MeTTaIL treats contextual stepping as authored operational authority. Belnap’s four-valued information states [8] motivate the two evidence orders of §19. Patterson’s constructible duality [9] is a substantially richer self-dual logic; the present formalization supplies only its two-lane certificate carrier and therefore avoids claiming an encoding or adequacy theorem for that logic.

22 Verification status

All results live in the Lean 4 tree under `Mettapedia/GSLT/LanguageDef/`:

Module	Content
<code>CertificateGSLT.lean</code>	objects; strict rule-retaining category
<code>CertificateGSLTOpen.lean</code>	open derivations; substitution calculus
<code>CertificateGSLTInterpretation.lean</code>	non-thin interpretation category
<code>CertificateGSLTSemantics.lean</code>	set-valued models; pullback; naturality
<code>CertificateGSLTIndexed.lean</code>	indexed functors; categories of elements
<code>CertificateGSLTOpenChecker.lean</code>	open-tree checker; exact reconstruction
<code>CertificateGSLTOpenDAG.lean</code>	chronological open-DAG checker
<code>CertificateGSLTDAGTransport.lean</code>	exact artifact transport
<code>CertificateGSLTRawSubstitution.lean</code>	raw substitution; cost accounting
<code>CertificateGSLTDAGSubstitution.lean</code>	sharing-preserving composition
<code>CertificateGSLTClone.lean</code>	multisorted-clone instantiation
<code>CertificateGSLTStepPresentation.lean</code>	proof-carrying trace generator
<code>CertificateGSLTStepAdequacy.lean</code>	fixture adequacy; modalities; budgets
<code>CertificateGSLTStepAdequacyGeneral.lean</code>	fragment gate; general adequacy
<code>CertificateGSLTStepPresentability.lean</code>	checker-format boundary
<code>CertificateGSLTWireFormat.lean</code>	versioned article wire semantics
<code>CertificateGSLTRuleJoin.lean</code>	fixed-core binary coproduct
<code>CertificateGSLTConstructibleDuality.lean</code>	paired evidence; Belnap orders
<code>CertificateGSLTInterpretationCanary.lean</code>	compiled positives and negatives
<code>CertificateGSLTDAGSubstitutionCanary.lean</code>	composition refuters; counts
<code>CertificateGSLTStepPresentationCanary.lean</code>	congruence and premise gates
<code>CertificateGSLTStepAdequacyGeneralCanary.lean</code>	gate falsifiers; binder canary
<code>CertificateGSLTWireFormatCanary.lean</code>	article mutation battery
<code>CertificateGSLTConstructibleDualityCanary.lean</code>	four-cell database
<code>MultiSortedClone.lean</code>	clone interface
<code>CertificateGSLTFramework.lean</code>	canonical import

The canonical import and the canaries build with zero warnings:

```
lake build Mettapedia.GSLT.LanguageDef.CertificateGSLTFramework \
    Mettapedia.GSLT.LanguageDef.CertificateGSLTInterpretationCanary \
    Mettapedia.GSLT.LanguageDef.CertificateGSLTDAGSubstitutionCanary \
    Mettapedia.GSLT.LanguageDef.CertificateGSLTStepPresentationCanary \
    Mettapedia.GSLT.LanguageDef.CertificateGSLTStepAdequacy \
    Mettapedia.GSLT.LanguageDef.CertificateGSLTStepAdequacyGeneralCanary \
    Mettapedia.GSLT.LanguageDef.CertificateGSLTWireFormatCanary \
    Mettapedia.GSLT.LanguageDef.CertificateGSLTConstructibleDualityCanary
```

There are no `sorrys`, no `admits`, no custom axioms, no wishlist stubs, and no `native.decide` anywhere in these modules. Axiom audits of the substitution laws, interpretation functoriality, semantic naturality, exact checker reconstruction, DAG transport, the strict-arrow negative theorem, the non-thinness witness, the sharing-count theorem, the general trace-adequacy theorems, the gate falsifiers, and the article correspondence and mutation theorems each report no axioms beyond `propext`, `Classical.choice`, and `Quot.sound`. Some algebraic results use fewer.

23 Limitations and future work

The current boundary is stated so that nothing beyond it is implicitly assumed.

1. **Shared judgment representation.** Interpretations preserve the common `Pattern` judgment IR. This suffices for independently authored proof systems compiled into the shared IR, but it is not yet a category of arbitrary syntax translations. The next layer adds syntax- and judgment-translating interpretations over the existing structural `LanguageDef` morphisms, with the same transport theorems.
2. **Interpretation of artifacts by rule templates.** Premise-hole substitution of checked artifacts is constructed (§13), with exact expansion and additive compact cost. What remains is pushing a whole artifact through a derivation-valued *interpretation* — each rule node replaced by a DAG template for its image — while preserving sharing; until then the framework does not claim that interpretation composition preserves DAG-node cost.
3. **Displayed organization.** The categories of elements of §9 are concrete indexed endpoints. A displayed category of proof presentations over compositional syntax GSLTs, with reindexing along actual syntax translation, remains to be built.
4. **Cost and provenance decorations.** Optional decorations over compact DAG evidence, kept strictly separate from semantic judgment identity as §12 requires.
5. **Metamath and MM0 instantiation.** Both through the unchanged generic checker, with source adequacy in the front ends and `mm-lean4` as the conformance partner.
6. **CeTTa/Prime pack compilation.** A verified correspondence between framework evidence and the runtime proof-pack format.
7. **Beyond the certified trace fragment.** General adequacy now covers the gated fragment of §15. Relation-query, freshness, and quantified premises await certified premise translations; collection patterns await a verified rewrite-modulo certificate; reflective matching and configuration-level operational states (where a step is not term-to-term) each need their own presentations and adequacy arguments. Each extension should arrive with its own counterexample-calibrated gate rather than a widened default.
8. **Semantic identity of articles.** The wire version of §17 names the artifact grammar only. Persistent and cross-process articles need an envelope binding presentation digest, language identity, and policy version to the ABI version; byte framing and cryptographic hashing sit below the proved

canonicity layer, and sharing-optimizing encoders remain untrusted producers.

9. **Constructible duality and graded evidence.** The present Boolean status carrier needs a judgment-level negation interpretation, connectives and semantics before it realizes Patterson’s logic. PLN-style evidence additionally needs graded lane objects and proved composition laws rather than a Boolean relabeling.

References

- [1] R. Harper, F. Honsell, and G. Plotkin. A framework for defining logics. *Journal of the ACM*, 40(1):143–184, 1993.
- [2] W. M. Farmer, J. D. Guttman, and F. J. Thayer. Little theories. In *Automated Deduction — CADE-11*, LNCS 607, pages 567–581. Springer, 1992.
- [3] F. W. Lawvere. Functorial semantics of algebraic theories. *Proceedings of the National Academy of Sciences*, 50(5):869–872, 1963.
- [4] J. Lambek. Deductive systems and categories II: Standard constructions and closed categories. In *Category Theory, Homology Theory and their Applications I*, LNM 86, pages 76–122. Springer, 1969.
- [5] N. D. Megill and D. A. Wheeler. *Metamath: A Computer Language for Mathematical Proofs*. Lulu Press, 2019.
- [6] M. Carneiro. Metamath Zero: Designing a theorem prover prover. In *Intelligent Computer Mathematics (CICM 2020)*, LNCS 12236, pages 71–88. Springer, 2020.
- [7] J. Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science*, 96(1):73–155, 1992.
- [8] N. D. Belnap, Jr. A useful four-valued logic. In J. M. Dunn and G. Epstein, editors, *Modern Uses of Multiple-Valued Logic*, pages 8–37. D. Reidel, 1977.
- [9] A. L. Patterson. *Implicit Programming and the Logic of Constructible Duality*. PhD thesis, University of Illinois at Urbana–Champaign, 1998.